Chapter 2

Distributed Shared Memory

A *distributed shared memory*(*DSM*) system is a (software) system that provides the abstraction of the shared memory across multiple network nodes. Most DSM systems provide memory read and write operations that are the same as normal memory access with the same performance when the addressed data is local and up-to-date. It can be further extended with locking, exception handling and so on.

The abstraction provided by DSM provides a means of sharing state in a distributed system that is similar to that used in non-distributed systems, supporting the shared state view of distributed systems.

In this chapter, we first consider DSM systems, their implementation and various extensions for supporting distributed applications. The discussion points out various limitations of this approach as a means of sharing state in a distributed system. We then consider specialized consistency programs and specialized "memory-like" interfaces that provide additional state semantics. This leads to considering the general spectrum of state sharing techniques, ranging from consistent shared memory to direct communication techniques. This spectrum provides a basis for understanding techniques of state sharing used in subsequent chapters.

2.1 Why DSM?

Distributed shared memory provides the abstraction of conventional memory shared across multiple nodes in a distributed system. All programmers are familiar with representing state in memory by the very nature of programming. Many are also familiar with sharing state through shared memory, either between processes as supported by the operating system shared memory or shared segment mechanisms, or by sharing state between concurrent/parallel threads in the same address space, as arises with on shared memory multiprocessors. For example, a common structure

for parallel programs is a *task queue* or *workpool*:

- a central queue of tasks to perform is maintained in shared memory.
- there is a shared data structure on which tasks perform actions.
- processors select a new task from the task queue when their previous task is completed, execute it, and return for the next task. An example could be HTTP requests in a parallel web server.

Standard locking/synchronization techniques are used for synchronizing on shared data by each task/process. This processing model relies on any processor/process being able to perform any task, and in particular, being able to access all portions of the shared state of the application stored in (distributed) shared memory.

Thus, a key argument for distributed shared memory is that it provides a familiar abstraction of state sharing, allowing programmers and even existing programs to adapt to the distributed environment with minimal difficulty.

Operating systems also use the shared memory abstraction. For example, the Unix operating system was extended to run on multiprocessors such as those built by SGI, the SUN Sparc multiprocessors, Sequent, Encore and so on. For these systems, Unix was modified to allow multiple processors to access the Unix data structures, stored in shared memory, with the primary modification being the addition of locks to synchronize access to these structures. The uniprocessor operating system data structures otherwise remain basically unchanged except for this synchronization. Using this approach on a parallel system, processes are automatically load-balanced across the processors, automatically migrating from one processor to another. Without consistent (distributed) shared memory, facilities such as distributed load balancing and process migration require extensive software to support.

Distributed shared memory arises somewhat naturally using modern operating systems in a distributed file system environment. That is, if two programs executing on different nodes map the same file for read/write into their address space, then this region is effectively (distributed) shared memory between machines if file caching ensures consistency. That is, an update by one process to the file is reflected in the file mapped by the other process, so if that portion of the file is mapped in the second process, the update appears in the second process's address space.

Without file consistency across network nodes, the above file access does not behave like a common file. (Nor does it behave like shared memory.) This lack of consistency is often a problem for file access, unless some form of explicit form of consistency is used, such as file-level or block-level locking with cache flushing. Thus, the ability to map a network file on multiple machines simultaneously either provides the basic DSM capability or else raises issues of consistency.

The next section defines consistency.

2.2 Shared Memory Consistency

A (shared) memory is *consistent* or *coherent* if there is a total ordering on updates that is consistent with that observable (by reads and writes and knowledge of the code) at each of the participating nodes. Intuitively, this means that, if one node observes (by reading) that X was updated to some value v1 before Y was updated to some value v2, no other node observed a different order.

Fig. 2.1 is an *event diagram* that illustrates a sequence of read and write operations that is inconsistent. In this diagram, the dashed arrow indicates the *happens before*

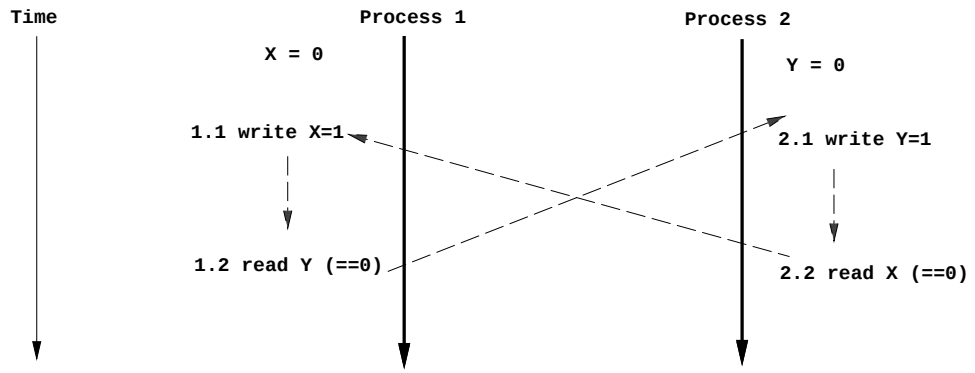

Figure 2.1: Inconsistent Ordering of Reads and Writes

relation between one statement and another. For example, statement 1.1 happens before statement 1.2. Assuming each process knows the code executed by the other, process 1 can observe that its statement 1.2 must have occurred before Process 2's statement 2.1 because it read Y as having a value of 0. However, process 2 can reason that statement 2.2 occurred before statement 1.1 in process 1 because it read X as 0. However, that implies the statement 2.1 must have occurred before 1.1 which implies 2.2 is before 1.2 as well, contradicting process 1's observation.

Inconsistent ordering such as this can arise if the writes are cached at each node and not propagated to the other node in time. In general, the "happens-before" relationship is defined based on the fact that writing variable with a given value always "happens before" that variable can be read to have that value, and that a process can observe the "happens before" relations among its own (sequential) actions. The "happens-before" relationship defines a *partial ordering* at each node in general. For consistency, there must be a total ordering that is consistent with the partial orderings observable at each process.

A conventional memory consistency mechanism ensures that there is a total ordering on the read and write operations performed in a distributed shared memory system

such that this ordering is consistent with that observed at each participating site. Each update is also performed in bounded time (and thus not lost).

Thus, consistency is a matter of ordering operations and a consistency protocol can be viewed as an ordering protocol.

2.3 Basic DSM Consistency Protocols

A DSM consistency protocol expresses consistency in terms of a shared object, such as a network shared file, following our earlier example, or a more abstract shared memory segment. Considering the underlying shared entity as a file, this file is mapped into a region of a virtual address space of each participant in the distributed shared memory.

Normally, the virtual memory page is the unit of consistency and transfer for this DSM. That is, one or more pages of data from the mapped file are cached for efficiency. When a page is referenced that is not present, the entire page is transferred locally from whereever the most recent copy of the page resides. When a locally modified page is provided to another node, the entire page is transferred.

The DSM module implements a readers/writer protocol on each virtual memory page so that there is either one copy of a page across the participating nodes that is writable, or no writable copies and zero or more readable copies.

Fig. 2.2 is a state transition diagram for a DSM page. Each arrow is labeled with the event causing the transition and (optionally) an action taken as part of the transition. The `ReadReq` and `WriteReq` are requests from the application (as a result of a read or write). A `r<X>` label indicates reception of a `<X>` message. A `s<X>` label indicates transmission of a `<X>` message.

This ownership protocol is also equivalent to a token-based sequencing protocol with extension to read/write distinction. In particular, if we consider the invalid, waitForExcl and Excl state only, a page is always owned exclusively by one node, the owner being the file server in the case of no clients holding ownership. When a node needs exclusive access to write the page, it requests this ownership from the current owner, which relinquishes this ownership to the requestor. Assuming the page starts out with one owner, it remains with one owner because a node always gives up the ownership as part of transferring the ownership to another node.

This simple exclusive mode would be sufficient for controlling reading and writing if both simply used exclusive ownership. However, it makes shared reading less efficient compared to allowing concurrent readers, which, if allowed, is not a problem for strict consistency.

With the shared reading extension, the last exclusive owner of the page can grant shared access to the page to each reader, maintaining knowledge of all readers in the

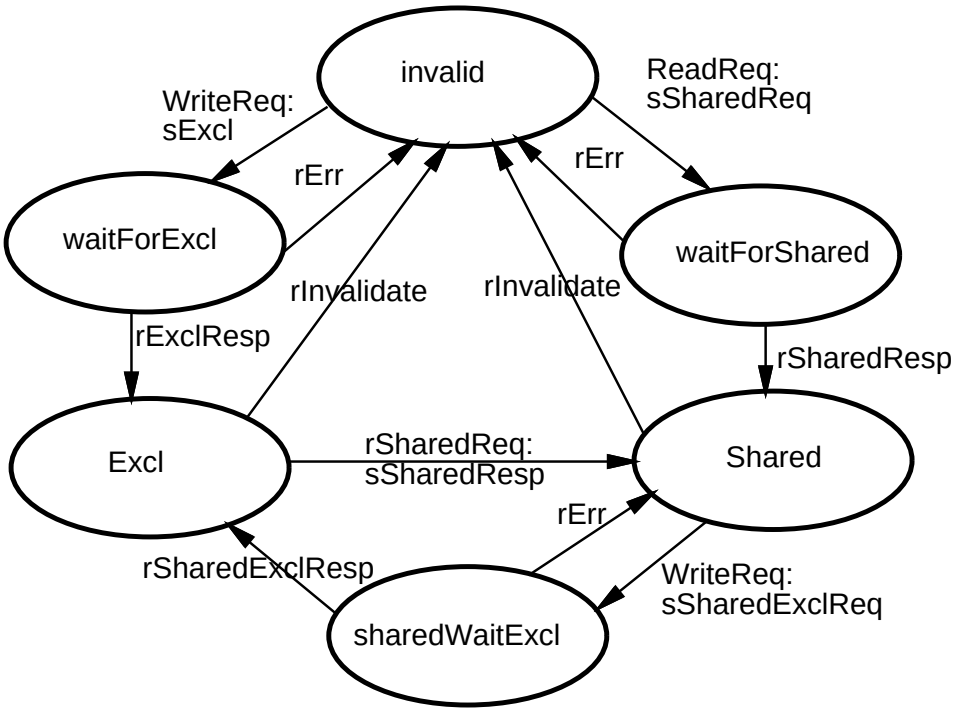

Figure 2.2: State Transition Diagram for a DSM Page

copy set. When the next writer requests exclusive ownership of the page, it needs to ensure that all nodes in the copy set of the page have invalidated their copies.

The DSM typically uses the virtual memory page protection to control access to these pages to ensure consistency. Thus, when a process attempts to write a page that is not locally writable, it incurs a protection violation and control is passed to the DSM. For example, in Unix, the DSM catches the `SIGSEGV` signal in its signal handler. The DSM then acquires ownership of that page from the current owner or invalidates the copies at all the readers, ensures the local page contents are current (by retrieving data from the last writer if necessary), enables writing on the page and allows the faulting process to continue. An attempt to read the page when the page is not readable causes similar handling, except the DSM places the page in shared mode. In response to a call from a remote node, the local node can provide a copy of a specified page as well as set the protection on the page to shared or none, depending on whether the call is requesting exclusive or shared access to the page.

The state machine representation of the protocol is a useful tool. It facilitates checking completeness because you can ask about all possible messages arriving in all possible states. In this sense, Fig. 2.2 is incomplete in numerous ways. For instance, there is no indication of what happens if a SharedReq arrives when the page is in the invalid state — an error indication is returned.

The state machine representation also helps in structuring the software. You define a basic protocol state around the states of this state machine and procedures that handle each of the possible messages, taking the actions specified by the state machine. This representation clearly indicates that protocol software is event-driven, rather than following a pre-specified flow of control, at least beyond the granularity of an individual state transition.

2.3.1 DSM Directory: Page Meta-information

A significant portion of the DSM implementation is maintaining a directory describing the state and current owner(s) of each page, so that the DSM can determine efficiently which node to contact on each fault. (This information is called the *meta data* for the page by some.) Specifically, the directory normally maintains a record for each page, with the record containing:

access: whether the page is readable or writable.

copy set: The nodes that have read copies of the page.

lock: Synchronization for multiple local processes accessing the same page and remotely requesting the page.

Logically, as part of the copy set, a page (owner) location is supported in the directory by providing a `probableOwner` field. When a process has a fault, it requests the page ownership from the node indicated as the probable owner. This value is just a cached *hint*. If this node has the page, the protocol proceeds as above. Otherwise, the probable owner can forward the request to a new owner, or can return an error indication to the requestor that causes it to multicast to locate the current owner. (The multicast can restrict the request to just the DSM participating nodes, rather than all network nodes. It may be feasible to use multicast to restrict the set of nodes further, such as using multiple multicast groups based on a hash of the page address. However, this last extension may not provide any savings, depending the page sharing patterns.)

The probable owner field is updated by a current exclusive owner to the requesting node when it turns over ownership of the page to a node requesting exclusive ownership. It is also updated to the requestor when an invalidation request is received when the page is locally readable. It is not updated when the ownership changes and this node is not an owner. Li and Hudak(89) show that a page location request that is forwarded always reaches the owner, (obviously after at most $N-1$ forwarding messages, where N is the number of participating nodes). They also observe that a relatively small number of nodes are generally accessing a given page, so the cost in reality is rarely more than 2 forwarding operations. For systems without a broadcast capability, they also propose a distributed copyset tree that allows invalidations to be propagated, ideally using $log(m)$ messages if a balanced tree, rather than broadcast.

The above description follows the basic *dynamic distributed manager* approach of Li and Hudak. They also describe a centralized scheme in which the page location information for all pages is located at a particular central node and a fixed distributed scheme in which the ownership of pages is partitioned across the nodes in some fixed way. However, their performance measurements of Ivy and experience with subsequent other systems suggests that the dynamic and distributed approach is more efficient, at least with well-structured parallel programs[1]. Implementation of distributed directories is considered further in Chapter 5.

2.3.2 DSM versus Shared Memory Multiprocessor

The basic components of a DSM implementation are similar to those of a shared memory multiprocessor (SMMP). A data cache provides local storage of copies of data that has been accessed locally. A backing storage area provides a home site for data to which it is written when removed from the cache after modification,

[1]The poorly structured programs perform so badly because of contention that the directory behavior is irrelevant.

and from which it is read on cache load. Finally, a data transfer and consistency protocol provides transfer between backing storage and the caches, and sometimes between caches.

However there are several significant differences. First, a DSM often provides optional partial or selective sharing between nodes via "segments" or files. In contrast, a SMMP normally shares all of physical memory, so the sharing is total and non-optional. Thus, a distributed application may use a DSM to share a portion of its distributed shared state, such as an infrequently updated but widely read directory, while the rest of the application uses other state sharing techniques, such as direct communication. A failure of a participating node may cause the application to patch the directory data structure and continue. However, with a SMMP, a failure of one cache or processor normally brings the entire system to a halt because the operating system data structures may have been corrupted in indeterminate ways.

Second, DSM has to deal with update messages that may arrive out of order whereas SMMPs, at least those interconnected by a single bus, can rely on the bus to serializing all the updates in the order they occur.

Finally, a DSM cannot depend on there being a reliable broadcast facility, whereas SMMPs usually use and require a reliable broadcast bus between processors. Consequently, a DSM must use a more complex protocol that handles all the normal problems of a transport layer including lost, corrupted and reordered packets. Moreover, reliable multicast is required to allow use of multicast to invalidate multiple shared copies of a datum when a write is pending, further complicating the transport protocol implementation. Note that the write update protocol used in some SMMPs is impractical in a network environment: it requires a reliable broadcast on each write to a shared memory cell. As noted in Chapter 1, the interconnecting network in distributed systems tends to be substantially slower in roundtrip latency and bandwidth, as well as having a much higher probability of error than the interconnection technology used in SMMPs.

Overall, you can regard a shared memory multiprocessor as the special hardware case of providing shared memory at the lowest level for a set of typically homogeneous processors commonly located on the same board or at least in the same box.

DSM presents some significant challenges in scaling distributed systems in the area of performance, heterogeneity and fault-tolerance/robustness. The next sections discuss these issues in turn.

2.4 DSM Performance and Program Behavior

Using DSM, a program written to run on a SMMP can, in theory, run on a cluster. However, the performance depends on the amount of contention (or consistency-induced traffic), especially with the typically larger unit of consistency used in a

DSM. (Actually, there is the cost of cache misses due to cold-start as well as replacement, but consistency is often the highest cost.)

A DSM often has a much higher cost for consistency for several reasons. The unit of consistency and transfer as well as mapping is normally a page rather than a cache line because the page is the minimal memory unit that the software can control access to on standard hardware. A virtual memory page is typically around 4 kilobytes in size (or larger) while a cache line is 16 to 128 bytes currently so a DSM has to move much more data when there is contention than an SMMP. This cost is further increased with a DSM by the longer roundtrip time between nodes, lower bandwidth and unreliability of the network, as noted above.

Even a moderate degree of contention can make the performance worse than running the program sequentially on one of the nodes because of the dramatically higher cost for the contention. For example, consider variables A and B stored on the same consistency unit, such as the typical 4 kilobyte virtual memory page used by many distributed shared memory systems. Suppose node 1 writes A, then node 2 writes B, and then node 1 reads A. A full page transfer occurs at least twice so the cost of 4 or more network messages is incurred. That is, the writer requests exclusive ownership of the page (and possibly the page contents itself) for two messages (including the acknowledgement), and the reader sends a request message and gets a response back granting shared ownership, and providing the 4 kilobyte page containing the modified variable. This whole sequence takes approximately 1 milliseconds on a 1000/100 Mbit Ethernet, versus less than 100 nanosecond if strictly local to a node, such as on a SMMP. The difference is greater than a factor of 10,000, which is roughly the ratio of a minute to a week.

Thus, with all but a very low degree of contention, all the benefits of parallel execution can be easily negated by the millions of cycles lost to consistency overhead.

The above example also illustrates the problem of false sharing. With *false sharing*, there is physical contention because of variables on the same consistency unit even though there is no logical contention. In the above case, A and B happen to be allocated in the same memory page, causing contention even though there is no logical contention, i.e. there is no concurrent access to A, nor to B. This problem is more common with the large consistency unit typically used with DSM than with small consistency units of shared memory multiprocessors.

The "solution" to this problem is carefully managed sharing, alignment of data to page boundaries, placement of shared data to reduce contention, and the use of message-like copying between areas "belonging" to different processors. However, this careful programming further detracts from the claims of shared memory allowing ease of development.

To provide reasonable DSM performance, one needs to implement DSM to provide finer-grain consistency units or (re)structure DSM programs to have very little

contention in spite of the large consistency units, or ideally do both.

2.4.1 Fine Grain Consistency

There are three units in cache:

- consistency unit — unit of update and ownership.
- addressing/mapping unit — unit of mapping and space assignment in cache.
- transfer unit — amount moved into cache, out of cache on misses, replacement, etc.

In a simple hardware system, all three are the same unit from the software standpoint, namely a virtual memory page. However, they can be handled as separate units. For instance, most computer architectures support a per-processor cache in which the cache line is the consistency and transfer unit. The ParaDiGM (Cheriton91) makes this cache line available to the software as a unit of consistency while retaining the page as unit of mapping.

Using the cache line as the unit of consistency rather than the page in a DSM should reduce the amount of false sharing, and reduce the cost of explicitly avoiding false sharing. That is, putting each variable in a separate consistency unit avoids false sharing but increases the fragmentation of memory, especially if the consistency unit is large. The small consistency unit also reduces the transfer time when the datum fits in this unit. However, if the unit of consistency is also the unit of transfer, a small consistency unit can lead to excessive transfer cost when the logical datum is significantly larger, such as a page. For example, if the unit of consistency in the DSM is a cache line yet the program is really working with page units of access, there are $pageSize/cacheLineSize$ message exchanges to handle the traps as the program accesses the full page (one cache line at a time).

Multi-unit transfers reduce the cost of small units of consistency assuming the system has some way to know the appropriate amount to transfer. Under software control, the transfer unit can range from a cache line to multiple pages, depending on the situation. The size to transfer can be an attribute of the memory region or of each page. For example an region of memory could be tagged as providing page-granularity consistency whereas another region could use cache line granularity, assuming the hardware and software supported both.

However, applications have to be programmed to use different regions with different consistency units, a daunting task given the difficulty of just getting memory management right in most applications. Moreover, this requires application (re)structuring to get this right, relative to conventional practice — further detracting from the primary claimed benefit of (distributed) shared memory.

2.4.2 Program Restructuring

The program (re)structuring optimizations for DSM include:

- Align each datum that is write-shared independently to the consistency unit boundaries to avoid false sharing, as above.
- Minimize/localize write-shared data, including structuring for processor/data affinity, so only contention when (re)load balancing objects among processors.
- Read-ahead to minimize latency effects.

However, aligning small units to page boundaries can result in a significant increase in space. For instance, aligning several 32-bit variables to 4 kilobyte page boundaries results in a 1000-fold increase in space for these variables. This increase can also slow down the program by increased paging. In the case of arrays and matrices, this alignment can require a non-standard array indexing mechanism (such as a C++ class), although these extra arithmetic instructions are typically not an significant extra overhead (if they do not entail extra memory accesses).

Localizing write-shared data often requires significant overall changes in the program structure, i.e. a rewrite, and cannot clearly always be accomplished, especially with matrix computations. There is also the danger that read-ahead introduces more false sharing effects. Also, it is difficult for the program to lookahead to prefetch a *sufficient* length of time relative to the high latency in getting the prefetched page. For example, consider an application fetching 128 byte application data unit (APU) over a 10 GE link that is using 1/4 of this bandwidth for this traffic with a 700 microsecond round trip time (because of queuing and processing delay in the network and at the other nodes). Thus, a prefetch is required at least 700 microseconds before the data is required, if my arithmetic is right, there can be 1600 APUs sent at this 2.5 Gbps data rate in this time so the application needs to prefetch 1600 units ahead. This is infeasible unless the APUs effectively form one or more *streams*, i.e. the APUs are pre-organized or specified as a stream of units so their sequencing is known far in advance.

Nevertheless, some applications, such as SOR and various physical simulations seem amenable to this type of restructuring, although not to the above degree.

Even with this restructuring, DSM can degrade badly if a page is actively write shared by two or more nodes. Mirage (FleischPopek89) uses a scheme in which a node can retain a page for at least some time period Δ. This technique is intended to reduce thrashing, but can also degrade performance if you do not understand the access pattern well.

2.4.3 Logical and Physical Contention Control

Locking can introduce additional physical contention when a second process tries to access a lock resident in memory that is already being accessed by some other process on a separate node. The act of reading the lock invalidates the page from the lock holder. The lockholder is then slowed in performing its computation and thus delayed further in releasing the lock, potentially leading to more contention and loss of performance.

As one approach, ParaDiGM includes locking support in the cache so that memory-based locking can be used to control cache contention as well as logical contention. In this scheme, the software is assumed to attempt to lock a portion of memory before accessing the data in that memory and the lock is assumed to use a test-and-set or compare-and-swap on a memory cell (byte or word). The act of acquiring the lock on the data sets a lock indication in the cache as well as writing the lock memory word in the cache line. If another process attempts to gain the lock on this same cache line, it will request an exclusive copy of the cache line as part of the lock operation. The ParaDiGM hardware causes the lock request to fail in this case, with the processor holding the lock not releasing ownership of the cache line. That is, the test-and-set or compare-and-swap appears to the software to have failed. This scheme can easily be extended to a DSM to limit unnecessary contention-induced traffic.

This same scheme can be extended to non-blocking synchronization techniques such as the load-linked/store-conditional used in some RISC processors. In this case, you want a conditional load from the version number that can fail if someone else has read but not written. Alternatively, you could provide a means to check whether your access has been preempted by someone else accessing the version number.

However, locking that is known to the underlying DSM implementation typically imposes complications such as requiring them to be specified on page or cache line boundaries or limiting the number of locks per memory unit. This requires further restructuring/restricting of the application software.

2.4.4 Heterogeneity

Most distributed systems contain heterogeneous computer nodes with different processors, byte-orders, different word sizes and basic data representations. With DSM, memory is shared between different nodes, yet is rendered meaningless if the representation of data type in the memory on one node is different from that on another participating node. For example, a page may contain double-word floating point values that need to be byte-swapped between one node and another. A DSM module could be extended to perform this conversion (with the attendant processing overhead) provided it also knew the types of the data in the memory, allowing it to

know what needed to be converted and how. The basic word or byte-level view of memory is not sufficient. It effectively needs to be a typed memory. The word-level access also does not provide a basis for knowing the types of the data stored in the word-level units being accessed.

The alternative of requiring a common data representation across all participating nodes seems excessively restrictive in general for a scalable system. Providing type information for all data in a shared memory page is extra programming complication, detracting from the primary DSM advantage. Furthermore, transforming types on page move is painful, especially if the data type changes in size.

2.4.5 Fault-tolerance and the Tight Coupling of DSM

Fault-tolerance is another issue with distributed shared memory. If a node fails holding the ownership of a page, the DSM either suspends a process requesting that page indefinitely until the node recovers (although a node does not typically recover with the page state intact), or else has to signal the failure to the process.

You can imagine providing a DSM-level exception or signal mechanism that allows an application to catch and act in response to these failures. For example, an application-specific handler may be able to determine that reverting to a previous version of the page after a failure may be satisfactory. Thus, the fault handler can arrange to acquire a copy and ownership of the page and then allow the process continue[2]. However, the author is not aware of any DSM providing this capability at the present time. Moreover, it seems hard to write an effective handler for a complex program, again detracting from the primary claimed benefit of DSM. For instance, if you are signaled a fault with some random address in a program, how would you determine what is stored there and how to handle the fault as a consequence?

The tight coupling of consistent shared memory limits the autonomy of separate nodes because the consistency protocol more or less requires nodes to give up ownership or invalidate copies of data on demand from other nodes.

2.4.6 Security

The tight coupling also raises some security issues because the state update process is so exposed, both in the sense of the update sequence being observable, and the updater being subject to waiting to acquire ownership of the datum it needs to update. Moreover, considering a client-server relationship, it is not sensible for the client to share its state with the server or vice versa, and it is difficult to program either to safely access a restricted shared segment without risk of corruption or compromise if the other party cannot be trusted. Of course, with the difficulty of

[2] It seems harder to control failure semantics with the distributed directory approach.

getting software right, it seems inadvisable to trust other software in any case if not necessary.

Thus, we conclude that providing a consistent shared memory between nodes of a distributed system does not scale well. Ease of programming essentially becomes a myth when the issues of performance, heterogeneity and fault-tolerance are considered. Performance suffers from the communication requirements of strict consistency, particularly with the long latency, limited bandwidth, large communication units of large-scale interconnection mechanisms. Heterogeneity complicates, if not renders unworkable, the low-level sharing provided by shared memory. Fault-tolerance suffers from the tightly coupled model of shared memory. Careful programming and restrictions of nodes to homogeneous assumptions allows DSM to be useful in limited contexts. However, the explicit handling of these issues eliminates the primary supposed benefit of the shared memory model. Conversely stated, one can view a shared memory multiprocessor as a tightly-coupled small-scale distributed system in which the processors and memories are homogeneous, the interconnection mechanism is very fast and specialized (such as providing reliable broadcast), and the entire system generally fails as a unit when any component fails.

Aside: MP3D I was involved in a project to restructure a wind-tunnel simulation program called MP3D which used the particle-in-cell approach to approximate hyper-sonic flow in a wind tunnel. The objective was to restructure this program to make it execute well on the ParaDiGM distributed shared memory multiprocessor. We aggregated particle and cell information in objects which were then made cache-line aligned by a specialized memory allocator. Furthermore, we provided processor affinity at the level of "precincts" to minimize contention and consistency traffic between processors. However, after all this work, MP3D still performed more poorly than expected and in fact degraded over time during its execution. After considerable effort, we traced this degradation to the fact that particles migrated randomly among pages in the course of the simulation, causing each processor to be accessing a much larger number of pages later in a run (and thus TLB thrashing). The only real solution seemed to entail copying particles between pages to maintain some degree of page locality relative to each processor and precinct. However, this copying is basically message passing, complicating the software further, and defeating any pretense of just being able to use shared memory.

These limits on conventional DSM prompt consideration of specialized consistency and specialized interfaces to shared state that introduce additional (helpful) semantics.

2.5 Specialized Consistency Protocols

Conventional DSM is far less efficient than message passing for some types of data access patterns. For example, consider producer-consumer access to data in which there is a cycle of one writer process writing the datum and another reading process (or set of processes) reading it. The direct communication approach requires two messages (including the ack) to "share" the datum whereas at least 4 are required in a conventional DSM.

With this observation, we can either give up on DSM for these cases or else incorporate multiple specialized consistency protocols in the DSM. Munin is an example of a system providing this latter option.

2.5.1 Munin and Treadmarks

Munin (Carter et al.89) is a DSM that supports multiple consistency protocols as well as the release consistency model of shared memory. With *release consistency*, shared data are only guaranteed consistent at synchronization points, e.g. end of critical section, i.e. at the release of locks. That is, it is the computer architecture term for (rediscovering) the synchronization that is used in atomic transactions.

With release consistency, a thread writing a datum does not have to wait until all the invalidations are transmitted for that datum because any other access to the datum would require locking the datum, which would synchronize with the current owner because of the locking, providing the required partial ordering on access.

Release consistency contrasts with *sequential consistency* in which each update is made available immediately. Release consistency requires that the synchronization mechanisms to be "visible" to the DSM, the Munin run-time in this case.

Munin provides 8 protocol parameters: invalidate or update, replicas, delayed op, fixed-owner, multiple-writers, stable-sharing, flush-to-owner, writable, all representing implementation decision points.

The programmer provides one of the following annotations on a Munin shared "object": read-only, migratory, write-shared, producer-consumer, reduction, result plus conventional sequentially consistent shared memory. Here, *merge consistency* combines the results of multiple writers on one consistency unit. This form of consistency is also supported in hardware by the Myrias system[3].

Performance: within 10 percent of hand-coded message implementations, for two applications measured, showing the power of the producer-consumer mode in particular.

[3]Myrias was a sophisticated multiprocessor developed in Canada in the 1980's.

One might question whether all the protocol variations are needed, given the programming and implementation complexity. Can programmers get it right? However, the annotations are performance optimizations and not required for correctness.

The merge consistency has been shown useful for matrix computations in which the writes from different processors to a shared matrix are interleaved on a word-by-word basis[4].).

Most of the performance benefits appear to come from the producer-consumer optimization, namely the traffic across boundaries in *SOR*(*Successive Over Relaxation*).

TreadMarks is a follow-on system to Munin that uses lazy release consistency rather than "eager" release consistency. (see paper for details). A variety of different consistency models have been explored in the research literature.

Another example of a specialized memory consistency is the ParaDiGM memory-based messaging, described below.

2.5.2 Memory-based Messaging

Shared memory can be used within a single node for communication. That is, a shared segment can be mapped into both the sender and receivers address spaces, the sender writes into its region that is mapped to the shared segment, and the receiver reads the message from its corresponding region. This behavior corresponds to the producer-consumer data access pattern recognized in Munin, which we recognized earlier as behaving very badly on a conventional coherent shared memory.

The ParaDiGM architecture provides an optimized form of memory-based messaging (CheritonKutter94) that relaxes the consistency to that assumed by communication, what is called *message-oriented consistency*. With message-oriented consistency, the receiver is only assured of seeing the latest update to that memory after it receives a signal on that memory. Moreover, that update and signal may be lost, i.e. the updates are delivered only with best efforts. Memory-based messaging optimizations also supports efficient address-valued signaling, which communicates an asynchronous signal from sender to receiver(s) indicating the address of the message. It also supports automatic signal-on-write at the end of a message (cache line), so the sender does not have to taken an explicit trap or call to signal.

In contrast to Munin, the memory-based messaging approach does not ensure conventional memory consistency, but fully optimizes for communication. Thus, the Munin approach is slower but safer, and the ParaDiGM approach is supportable in hardware and thus faster, but does not provide conventional consistency. The

[4]Large matrix computations may be the primary motivation for tightly coupled shared memory multiprocessors. They seem like the hardest, or perhaps least possible, to restructure to run efficiently on a DSM otherwise.

ParaDiGM approach seems better when the memory is used by a module that provides communication services and there is no danger of inappropriate dependence on consistency. The Munin approach seems preferred when the memory is fundamentally shared and the data access pattern just happens to be (mostly) producer-consumer.

The above illustrates some efforts to come up with more efficient specialized consistency protocols. Another approach over conventional DSM is using a specialized interface to the state — getting beyond just basic memory read and write.

2.6 Specializing the Memory Interface

A distributed shared memory can be regarded as a service that supports a simple read/write interface — simple and unstructured. An object-oriented view of this is

```
interface DistributedSharedMemory {
public:
    Datum read( Address ) const;
    void write( Address, Datum );
    . . .
  };
```

This could be extended with additional operations for reading and writing different sizes and various sizes of data. However, it remains the case that the data being read and written is only known to be basic types such as byte, word, double-word, etc.

In contrast, objects in general normally have far more specialized and semantically rich interfaces. As we have advocated with object-oriented programming, these interfaces may be entirely in terms of reading and writing *attributes*, logical abstract variables in the interface. However, these attributes provide specific type information and application-specific semantics to their operations.

Linda (CarrieroGelernter86) is an example of a specialized "memory-like" service, providing *tuple space*, which can be regarded as a structured distributed shared memory with specialized semantics (although not strictly application-specific).

2.6.1 Linda

A *tuple* is an ordered sequence of values, e.g. ["task1", 43, false] A *tuple space* is a set of tuples, a structured shared state space. In Linda, the tuple space is effectively a distributed content-addressable memory (CAM) of tuples. (More recently, Linda-like tuple spaces seem to have been adopted for use in the Jini Java-based distributed framework, originating from Sun.)

The primary Linda primitive operations are:

out — write tuple to tuple space.

read — read tuple space, blocking if no matching tuple.

in — same as read but also removes tuple from tuple space.

A tuple space can have multiple tuples with same "key", so there is no conventional consistency. As an alternative view, tuple space can be considered a collection of distributed queues, each queue being addressed by key. Thus, each key represents a reliable communication channel, often using *subject-based addressing* in the form of the key.

This characteristic is illustrated by an common example use of Linda, namely as a distributed task queue. A process in a distributed parallel computation determines the need to compute some function `f` on some parameter `arg` and so calls

```
tuplespace->out( ``compute'', f, arg )
```

to add this task to the tuplespace. An idle worker process on another node calls

```
tuplespace->in( ``compute'', var f, var param )
```

to receive a new compute task to perform and then calls

```
tuplespace->out( ``fresult'', result )
```

to make the result available in the tuplespace. At some point, another task needing the result of computing f calls

```
tuplespace->in( ``fresult'', result )
```

to get the computed result.

There are three main implementation approaches:

1. replicate tuple space on every node. In this case, reads are fast, outs and ins are expensive.
2. Hash tuples to sites. Then, outs are fast but reads and ins fast only if good match between hash and access patterns.
3. Store out at local site. Reads are slow unless most sites "in" their own "outs", which may not be the case with the above example.

The Linda tuplespace appears like an interesting and useful distributed object to have available for sharing state across multiple nodes as part of building distributed applications. (However, it does not seem necessary to view it as a separate language or system, as it was originally conceived.)

Another example is a directory service, with an interface as below.

```
interface Directory {
public:
    void newName( Name, Address );
    void deleteName( Name );
    Address address( Name ) const;
    . . .
  };
```

With this interface, we know that we are either adding a new name to the directory, removing a name or looking up a name. These are far higher-level operations than just reading or writing memory. We can take advantage of these semantics by recognizing that some level of inconsistency may be tolerated in the directory. That is, a new name can be added in one end of the network and a subsequent name lookup for this new name in a separate part of the network may fail shortly thereafter with limited consequences. It is normally sufficient to have the directory achieve *eventual consistency* so names are present within the time period that new names are normally distributed.

Other examples of shared state with semantics include:

- synchronized time
- game state of distributed game
- file and database contents

In general, a shared state module provides operations to read and write the state, mechanisms for locally caching data from other nodes and transmitting local updates to other nodes, and some type of consistency mechanism to ensure the state is "shared", that is, sufficiently consistent to provide common view of the state of the system. A higher-level interface provides semantics to exploit in this sharing, as compared to basic DSM.

The above examples illustrate the value of defining a higher-level interface to shared state, providing additional semantics and type information, supporting efficiency, heterogeneity and even fault-tolerance. The value of these additional semantics argue for a higher-level object-oriented view of this nature, rather than the low-level view of state as raw memory, as provided by DSM. The additional semantics and the specialized consistency protocols lead to the notion of a spectrum of state sharing techniques, as described next.

2.6.2 Javaspaces, OpenSpaces and Ramcloud

Java/Sun came out with Javaspaces some time ago, which provided a specialized stateful storage of a restricted set of Java objects, derived from a special Entry class.

In the past 5 years, there have been several commercial efforts to support an "in-memory data grid", with Gigaspaces.com being one such company. Terracotta.org is another example. There is also an OpenSpaces, providing open APIs to the Gigaspace version. My understanding is that both focus on providing a RAM-based cache of state that can be accessed from Java (with other languages supported), with extended support for event notification, transactions, message queuing, etc. In this vein, it is really more a specialized "memory", specialized with object interfaces and additional semantics.

Ramcloud is a project at Stanford to explore the notion of keeping all state in DRAM to improve performance and reduce the complexity and unpredictability of dealing with disk storage. It is unclear at this stage what data model they will use. However, it seems more likely to be objects and not basic memory read/write.

Overall, these approaches are more in the direction of type/object specialized state support and specialized consistency mechanisms than DSM.

2.7 The Spectrum of State Sharing Techniques

There is a spectrum of state sharing techniques, ranging from one extreme of direct communication techniques to the other of consistent shared memory, with application-specific or problem-oriented techniques as a middle ground between these two extremes, as suggested in Fig. 2.3. This spectrum trades off performance

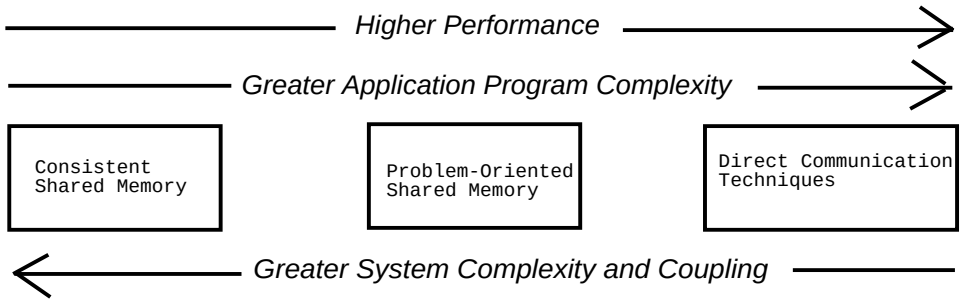

Figure 2.3: Spectrum of State Sharing Techniques

with application complexity. The next subsections discuss these techniques further.

2.7.1 Direct Communication Techniques

In the direct communication approach, the source of an update "shares" the state for the interested recipients by explicitly sending them a message containing (new) state or the update. Often, there are basic primitives of send and receive provided by the communication system for direct use, such as UDP datagrams, TCP connections, etc. (The human-level equivalents include e-mail messages, talk, telnet and so on.) Providing this basic facility as a remote procedure call rather than messages is simpler to use and ensures more error checking, but does not change the fundamental state-sharing technique. That is, the source still needs to explicitly make calls to the receiving objects.

The key advantages of the direct communication approach is that it is simple in simple cases and provides very fast transmission to the recipients, given it maps directly onto the network delivery service, as illustrated in Fig. 2.4.

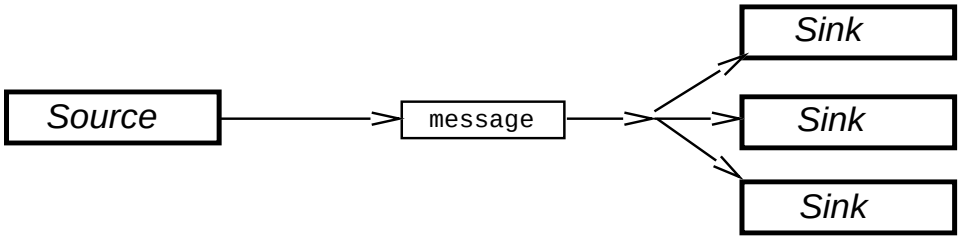

Figure 2.4: Direct Communication of Shared State

However, several restrictions arise with this approach in many distributed systems scenarios:

- The source must know the recipients or interested parties associated with the update.
- The sender and receivers must generally co-exist. For example, one cannot send a message to a process that does not exist yet but will be interested in the state update when it does.
- The sender does not know how often to send updates, or which receivers are necessarily interested in each update.

Another problem is that direct communication of updates often introduces ad hoc messages that are object-specific and error-prone. In particular, there is no clear or obvious coupling between the data structures at either the sender or receiver ends and the message or procedure call form.

The above describes the *push model* of communication. The last problem can be addressed to some degree by adopting a *pull model*, in which the recipients have to explicitly request each update from the source, also called *request-response communication*. Then, the "receivers" need to know about each source of updates and incur the cost of polling for updates, wasting resources if updates are infrequent with some source. The pull approach also introduces the round-trip time delay to get the latest value when this is required.

2.7.2 Consistent Shared Memory

The problems with direct communication often cause the communication primitives to be used to access shared state service, such as a file server, as suggested in Fig. 2.5. Here, the sources and sinks only need to know about the shared state

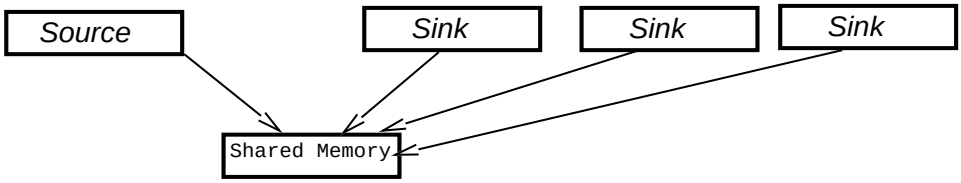

Figure 2.5: Indirect Communication Through Shared State

server, these sources and sinks need not co-exist simultaneous because the updates are held in the shared state server, and the negotiation of push versus pull, rate of update, polling and so on can all be aggregated into the shared state server, rather than between each pair of source and sink. In that vein, the shared state server can notify interested parties of updates using a (logically) multicast to this group, if the group is known. The storage model of the shared state server encourages a stable representation of data at the server that the source transforms its updates to, and the sink transforms the updates from. For example, a file format is such a representation, used with a file server as the shared state server.

Taking this shared state server approach, data caching is often of such enormous performance benefit in many applications, it *forces* the addition of caches, as depicted in Fig. 2.6. We use the term *forces* because caching is often so beneficial that it is necessary to achieve competitive performance, i.e. a mandatory aspect of a real system design.

The addition of caches produces duplicates of the state. That is, a datum can be stored in the shared state server as well as in one or more caches. A consistency protocol is required to keep the view of the shared memory consistent from each node, as described in the earlier discussion of DSM.

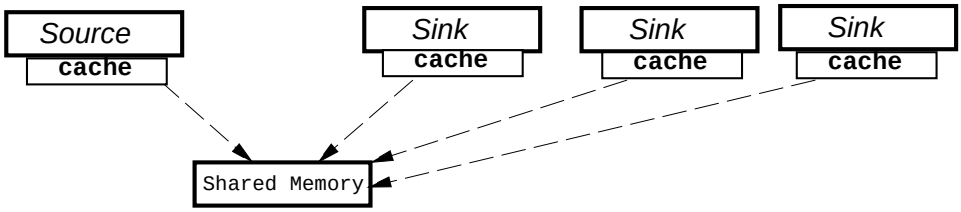

Figure 2.6: Cached Indirect Communication Through Shared State

Considering the advantages and disadvantages of these two models, a compromise approach is to provide a shared state model, but with relaxed requirements on consistency, particularly exploiting application-specific or problem-oriented requirements, as discussed below.

2.7.3 Application-Specific Shared Memory

The term *application-specific* memory (also called *problem-oriented shared memory*) refers to a module providing a (shared) state abstraction i.e. read and write operations on shared state) but using specialized consistency that exploits application semantics. In an object-oriented distributed system, this approach means that the distributed object consistency for a particular distributed object class may use class-specific consistency techniques.

An application-specific interface defined by object-oriented techniques provides additional semantics that can be exploited for reducing the cost and complexity of consistency as well as increasing fault-tolerance. These specializations are illustrated in the following examples.

Real-time Video

Here, the *shared state* is the video stream itself. Given its real-time nature, this state is best provided in general by a "pushed" multicast stream over the network. The proxy object on the client can provide a *playback buffer* which the display process can simply read on each frame time. The playback buffer hides the communication details from the display process, including the jitter arising from network delivery. There is no need for conventional consistency because the consumer of the state, the display process just needs the new state from the state and it is accessed sequentially.

This shared state is updated with best-efforts delivery, recognizing that a packet loss is recovered from by the update sent for the next frame time, at least with raw video. With compressed video, loss of a I-frame can corrupt the output for a longer time, but only until the next I-frame. (TCP Fast retransmit can often recover a lost packet in time to avoid a frame loss while not delaying the video playback longer that provided for by the playback buffer. TCP can be easily modified to allow recovery within a bounded period of time or else continue over the missing segment (ref. TCP-RTM)).

Amaze: A Distributed Multiplayer Game

In this computer game (BerglundCheriton85), "monsters" move in smooth animation through a maze, shooting at each other. Players can join and leave game at

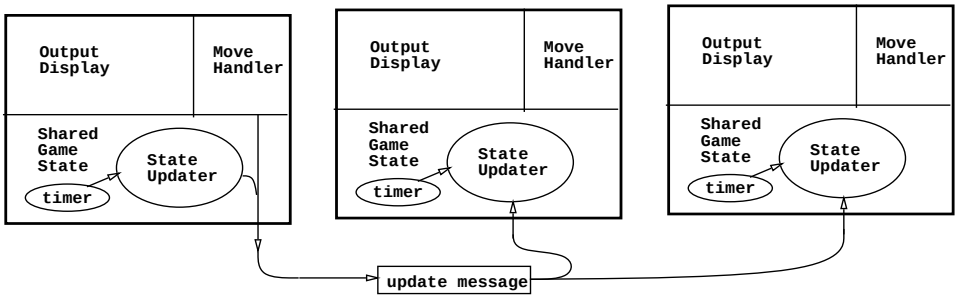

Figure 2.7: Amaze Multiplay Game

will. As suggested in Fig. 2.7, the game state is replicated at each node, each running an instance of program. Each local node evolves state forward in time, using velocity of monsters, missiles, etc., a technique referred to as *state extrapolation.* (This approach is also referred to as *dead reckoning.*) The nodes communicate periodic corrections of position plus updates that come from move handler. That is, it sends "2nd derivative information", i.e. changes to 1st derivative, plus the absolute monster positions for position correction.

Using this approach, shared game state is *sufficiently consistent* across the participating nodes as long as update messages are received within certain periods of time. By sufficiently consistent, we mean that the difference between the perceived location of a monster and the actual location is insignificant to the game play. The replication of the state means that failure of one instance of program only effects that player, and does not delay or terminate the rest of the game.

This general approach of actively evolving the state forward using application-specific semantics is also applicable to clock synchronization, where the time is the shared state, and also distributed scheduling, where processing resource availability is the shared state.

These topics are addressed further in Chapters 4 and 11, respectively. This approach is also applicable to distributed interactive simulation and distributed robotics.

Real-time Factory Control

The real-time factory control system may be distributed across sensor computers and controllers and storage servers to colocate real-time tasks with the equipment and to allow for fault-tolerance, reconfiguration and extensibility. As one aspect of the operation, an oven controller may need to know the reading at a particular sensor, such as an oven temperature, which is changing over time. Using the application-specific shared memory approach:

- The controller has a local object indicating the "current" reading of the sensor.
- The sensor periodically multicasts an update of the reading to controller.
- The controller variable is *sufficiently consistent* if it received at least one of the last (say) 5 updates.

In an object-oriented implementation, the sensor would appear as a local object to the controller, and the communication required to keep this local object for the sensor sufficiently consistent with the real sensor would be encapsulated in the (distributed) sensor object implementation.

With this structure, reading of the sensor by the controller software is a local object operation decoupled from network operations and the real sensor updates. The local

sensor object simply relies on the multicast updates to ensure the value is sufficiently consistent on each read. Given the real sensor can be changing continuously, strict consistency is not provided, but this application-specific consistency is sufficient and much less expensive.

The local sensor object can be specialized for its semantics in other ways too. For example, it can use the average of readings from several sensors, possibly averaging the results in some application-specific way. Commonly, the sensor readings are fed through a Kalman filter which produces a reasonably accurate result even in the presence of noise in the sensor readings. There can also be multiple controllers monitoring this sensor or set of sensors, each receiving the sensor multicast updates. These extensions for fault-tolerance all rely on specific semantics and would not be feasible using conventional consistent shared memory for state sharing.

This structure contrasts with the direct communication approach in which the controller software explicitly deals with the communication primitives. It also contrasts with the consistent shared memory model in which consistency traffic and delays would occur on most read and write operations.

This example also illustrates another aspect of application consistency that arises in real-time systems. A real-time control system monitors the state of the "real" system, i.e. the factory. Consistency with "real" system, what you might call *temporal consistency*, is almost always more important than strict internal consistency. Having the controller read a recent update from the sensor is generally preferred over having the strict consistency protocol delay that update, such as delaying the sensor's write operation until the read completes at the controller and exclusive ownership is transferred back to the sensor node.

NFS

The SUN Network File System (NFS) is another example of an application-oriented shared memory in the sense that it provides a basic shared state model but does not provide strict consistency. That is, state is accessed as files through the open/read/write/close file interface. NFS "consistency" is provided by timing out pages in the cache so a page can be at most t seconds out of date, for some parameter t (which is often set at 30 seconds).

This form of consistency is viewed as sufficient because there is relatively little write-sharing of files between different applications, especially those running on separate network nodes. However, one can easily observe the resulting inconsistencies by cycling between editing a file on one node and then immediately compiling it on another node that is NFS mounting that file system. The second machine may still be caching the previous version of the file and thus compile an old version of the source file.

Transaction State

A transaction is a distributed object that represents a collection of state updates, to be performed atomically. The transaction state follows a particular sequence, namely inProgress, prepareToCommit, commit, abort The state progresses forward through these states, stopping at commit or abort (and never going back). Transaction-specific mechanisms are used, rather than DSM techniques, to ensure that all participants in a transaction take consistent actions, even though they are not necessarily updated precisely as the state of the transaction changes.

See Chapter 7 for details.

2.8 Concluding Remarks

Distributed shared memory (DSM) is a low-level shared-state mechanism for distributed systems. It can be implemented to provide the abstraction of shared virtual memory, the same as available on the shared memory multiprocessor. A basic DSM implementation is fairly straight forward. However, a basic implementation exhibits poor performance if there is more than a minimal amount of contention. The contention is increased in amount and cost by the typical large size of the consistency unit used in DSM, namely a page. Careful program structuring can reduce contention, especially that induced by false sharing. However, further optimizations require extensions to the basic DSM implementation.

There are a variety of optimizations one can apply to DSM. Munin and Treadmarks explored the use of so-called release consistency. Munin also explored the use of a variety of consistency protocols. The one of primary benefit, that supporting the producer-consumer access pattern, was noted as a step to memory-based messaging support. These cases and other specialized memories such as illustrated by Linda further suggest a general theory of shared state maintenance.

However, DSM has dramatically limited applicability because of its failure to provide isolation for fault-tolerance and security, and its difficulty in dealing with heterogeneity of machines. Moreover, except for a few very partitionable or small-scale applications, the complications to make DSM perform well appear to negate the claimed benefits of familiarity and ease of programming of the (distributed) shared memory model. This reinforces the message left behind by Allan Turing, namely: "I showed you could do make (almost) anything work, the challenge for you is efficiency!"

In general, the challenge is providing scalable state sharing and the problem is consistency, in performance, fault-tolerance and security, and as well as system complexity and heterogeneity. There is a spectrum of state sharing techniques, with application-specific shared memory arising as the compromise between direct

communication and strictly consistent shared memory, suggesting examining specialized application-specific consistency protocols. In the object-oriented view of the two extremes, a consistent shared memory is the most generic application-semantic-free implementation of an object, in terms of state. That is, it can be written and read in terms of basic memory units and is tightly coupled to provide strict (general-purpose) consistency at that level. At the other extreme, the communication channel can also be viewed as an object with extremely loose consistency on it. Namely, a write or send on the object may or may not make this same data available at the recipient (assuming a best-efforts but unreliable channel), and the update is not synchronized with the reader or receivers, except possibly by flow control and sequencing. The messages often carry application-specific messages. The compromise middle position of using application semantics provides much of the conceptual simplicity of the shared memory abstraction but much of the performance and fault-tolerance of the direct communication approach.

A number of examples of state sharing and state sharing protocol techniques come up in the following chapters as we consider particular services.

The next chapter explores structuring distributed systems from an object-oriented standpoint and its implications.

Readings

Amza et al., TreadMarks: Shared Memory Computing on Networks of Workstations, IEEE Computer, 1996.

References

K. Li and P. Hudak, Memory Coherence in Shared Virtual Memory Systems, ACM Trans Computer Systems, Vol. 7, No. 4, Nov. 1989, p. 321-359.

J. Bennett, J.B. Carter and W. Zwaenepoel, Munin: Distributed Shared Memory based on Type-specific Memory Coherence, 2nd ACM Symp. on Principles and Practice of Parallel Programming, March 1990, p. 168-177.

J.S. Chase et al. The Amber System: Parallel Programming on a network of multiprocessors. Proc. 12th ACM Symp on Operating System Principles, p. 147-158, December 1989.

R.G. Minnich and D.J. Farber, Reducing Host Load, Network Load and Latency in a Distributed Shared Memory. Proc. 10th Int. Conf. on Distributed Computer Systems, IEEE 1990, p. 468-475.

N. Carriero and D. Gelernter, Linda in Context, Comm. ACM, Vol. 32, No. 4, April 1989, p. 444-458.

N. Carriero and D. Gelernter, The S/Net Linda Kernel, ACM TOCS, May 1986.

A.S. Tanenbaum, F. Kaashoek and H. Bal, Using Reliable Broadcasting to Implement Distributed Shared Memory Efficiently, June 1990.

B. Fleisch and G. Popek, Mirage, SOSP'89.

D.R. Cheriton and H. Goosen and P. Boyle, ParaDiGM: A Highly Scalable Shared-Memory Multi-Computer Architecture, IEEE Computer, 24, 2, February, 1991.

D.R. Cheriton and R.A. Kutter, Optimized Memory-Based Messaging: Leveraging the Memory System for High-Performance Communication, submitted for publication, Stanford CS Technical Report, 1994.